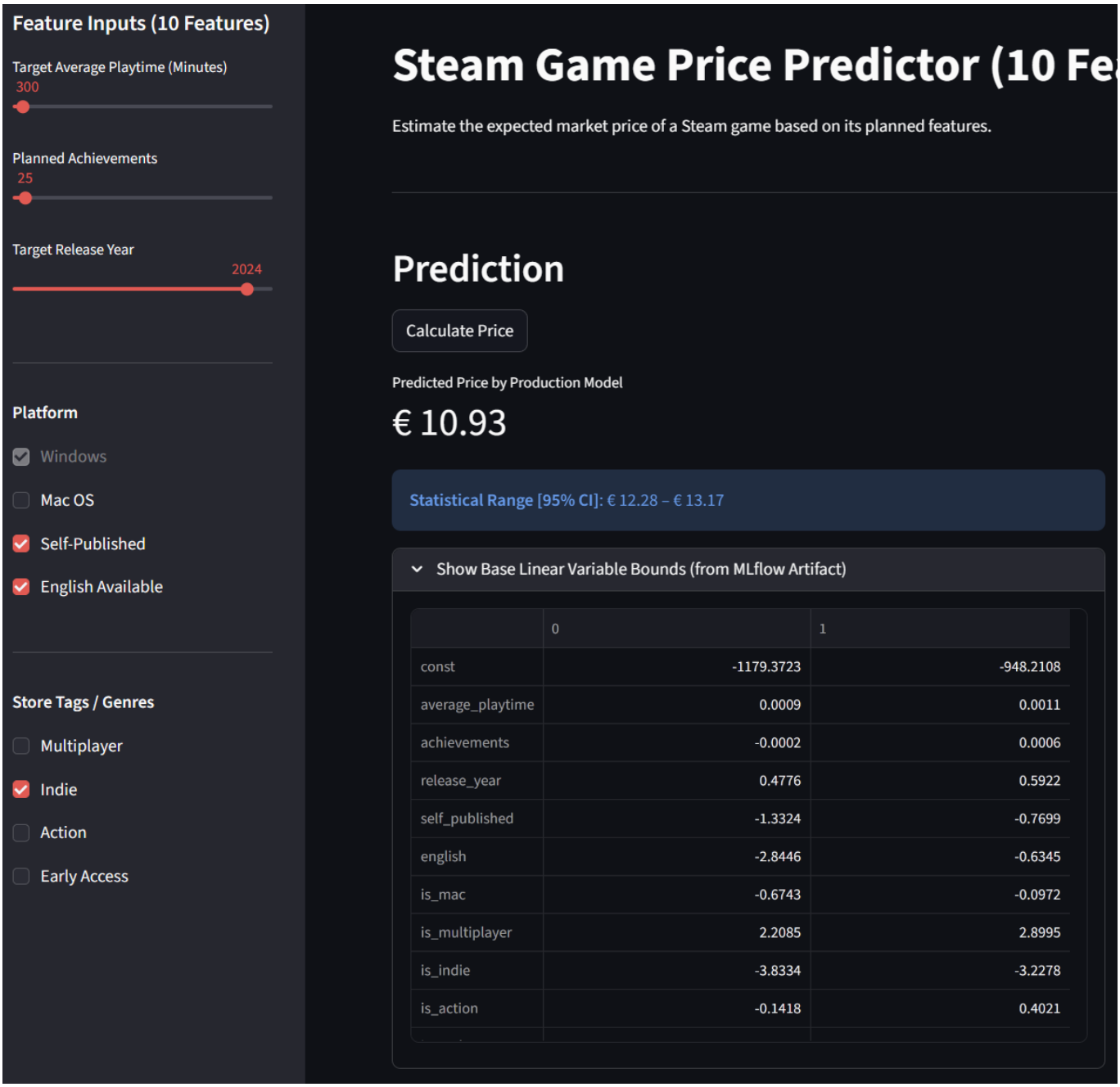
Phase 3: Interactive Dashboard with Streamlit

1. Screenshots of the Running Streamlit App

The screenshots below show the Streamlit dashboard (`http://localhost:8501`) running `BestRegressor_v1` (GradientBoostingRegressor, Version 11) loaded from the local MLflow Model Registry.

1.1 – Sidebar Inputs, Prediction Output and Confidence Interval

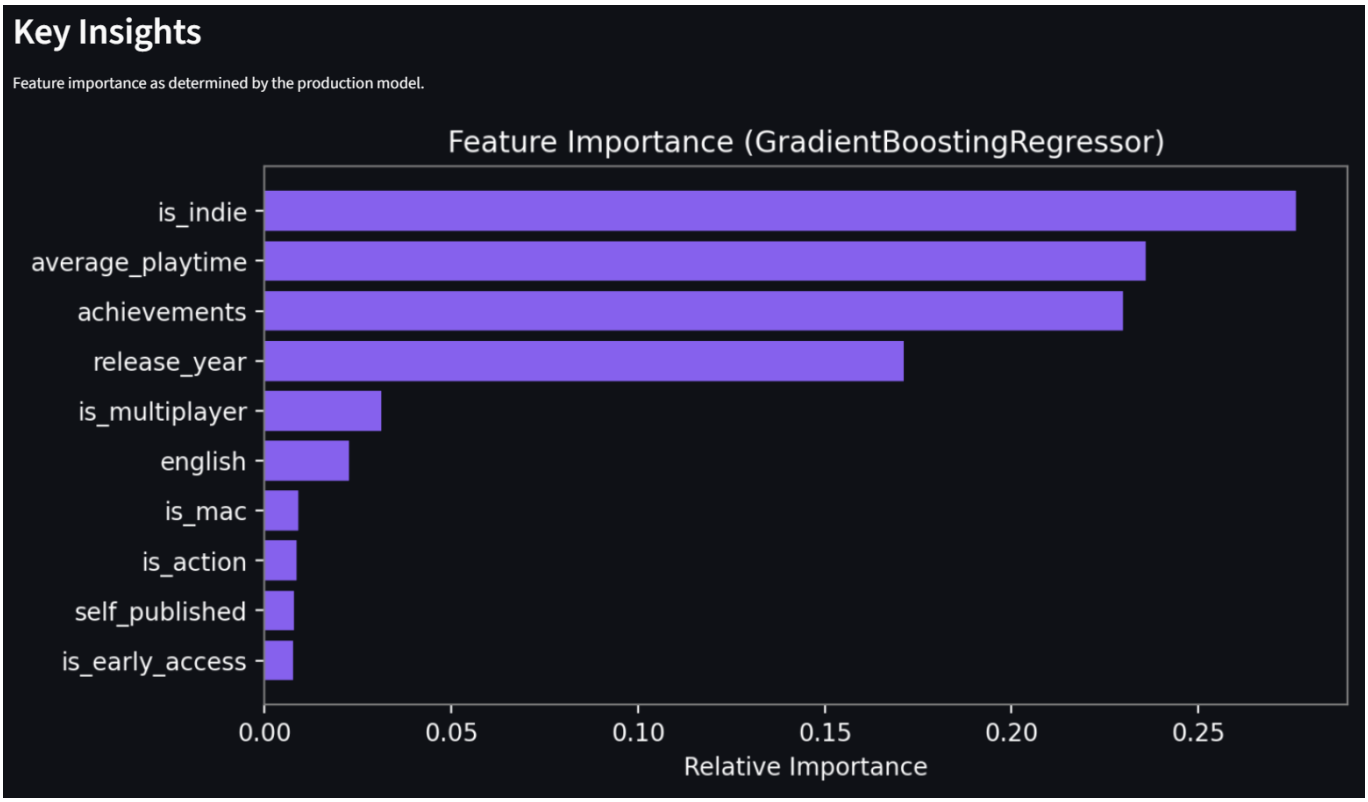
The left sidebar provides all 10 feature inputs: three numeric sliders (playtime, achievements, release year) and seven checkboxes (platform, publisher, genres). After clicking **Calculate Price**, the dashboard shows the predicted price from the production model and the 95% confidence interval calculated via `statsmodels`. The expandable table shows the raw coefficient confidence intervals loaded as an artifact from MLflow.



	0	1
const	-1179.3723	-948.2108
average_playtime	0.0009	0.0011
achievements	-0.0002	0.0006
release_year	0.4776	0.5922
self_published	-1.3324	-0.7699
english	-2.8446	-0.6345
is_mac	-0.6743	-0.0972
is_multiplayer	2.2085	2.8995
is_indie	-3.8334	-3.2278
is_action	-0.1418	0.4021

1.2 – Key Insights: Feature Importance

The right panel shows a horizontal bar chart with the relative feature importance of the GradientBoostingRegressor. `is_indie`, `average_playtime`, `achievements`, and `release_year` are the dominant predictors.



---

2. Architecture Overview

Model Loading

The Streamlit app connects to the local MLflow server (`http://localhost:5000`) and loads `BestRegressor_v1/Production` via `mlflow.sklearn.load_model`. The `@st.cache_resource` decorator ensures the model is only loaded once on startup and reused for all subsequent requests, avoiding repeated reloads on every input change.

The app also downloads the `conf_intervals.csv` artifact from the same production run, making it available in the expandable confidence interval section.

Execution Flow

1. The user adjusts inputs in the sidebar (sliders and checkboxes).
2. Streamlit assembles the inputs into a single-row DataFrame with 10 columns matching the model's training schema.
3. On button click, `model.predict(input_data)` returns the estimated price.
4. **(Bonus)**: A cached `statsmodels` OLS model is used to compute the 95% confidence interval via `lr_sm.get_prediction(sm_input).summary_frame(alpha=0.05)`, displayed beneath the prediction.

---

3. Discussion of Key Insights

The feature importance chart shows the following ranking:

Feature	Relative Importance	Notes
`is_indie`	~0.27	Strongest predictor — indie games are consistently cheaper on Steam
`average_playtime`	~0.23	Longer playtime correlates with higher production level and price
`achievements`	~0.22	More achievements generally indicates a larger, more polished game
`release_year`	~0.17	Newer releases tend to be priced higher
`is_multiplayer`	~0.03	Multiplayer support adds some price premium
`english`	~0.02	English availability slightly correlates with internationally released titles
`is_mac`, `is_action`, `self_published`, `is_early_access`	< 0.02 each	Minor individual contribution, kept for combined effect

Why GradientBoosting Outperformed the Other Models

Gradient Boosting builds trees sequentially, where each tree corrects errors from the previous one. This allows the model to improve on cases it initially gets wrong, such as mid-tier games in the €30–60 range, where features alone are ambiguous. This results in a lower RMSE compared to Random Forest and the linear models.

Possible Improvements

- Adding a publisher-tier feature (e.g., whether the game is from a major publisher) could help separate budget indie games from higher-priced studio releases.
- A text feature based on the game's store description could capture genre and content information not covered by binary tags.